In [ ]:
```
import marimo as mo
import matplotlib.pyplot as plt
import numpy as np
```

Turbo fan remote: CR2032 battery life

How much of a CR2032's capacity the remote can actually use, and how long that lasts, for two settings of the in-frame brown-out abort level (`PVD_LEVEL_ABORT` in `fw/remote.c`).

Why the cut-off matters. The firmware stops transmitting when either:

1. VDD *at rest* (MCU awake, LED off) is below the "OK to transmit" PVD level, or
2. VDD *during a frame* dips below the abort PVD level.

A coin cell's internal resistance climbs steeply near end of life, so the remote stops working before the cell is actually empty. Raising the abort level by about 0.2 V gives more margin above the MCU's 1.8 V minimum but gives up some capacity.

The PVD only has discrete levels, so "1.87 V + 0.2 V" becomes the nearest level, **2.08 V** (PLS = 010, falling edge).

Sources

What	Source	Used for
CR2032 cell	Energizer CR2032 datasheet (form 2032NA0618)	Rated capacity, 235 mAh to 2.0 V at 15 kΩ
CR2032 behaviour	Energizer Lithium Coin Handbook (2018)	p5: OCV and internal resistance vs capacity, 75 Ω pulses (~30 mA, 0.1 s on / 4.9 s off). p4: 0 °C vs 21 °C at 1 mA. p7: ~1 %/year self-discharge, 10-year shelf life
MCU	WCH CH32L103 datasheet (local copy: `datasheets/CH32L103DS0.PDF`)	Run/standby current (Tables 3-6, 3-8-1), PVD levels and PDR (Table 3-4), reset duration
IR LED	Vishay TSAL6200 datasheet (doc 81010)	I–V curve (Fig. 4, 100 µs pulses), radiant intensity vs current (Fig. 5), −1.8 mV/K
LED reservoir caps	Samsung CL31A107MQHNNN characteristics, reference spec sheet, Samsung product page, LCSC C15008	DC bias −40 % at 3 V; insulation resistance ≥ 50 MΩ·µF

The handbook's p5 curves suit this design better than the datasheet's pulse chart: they come from ~30 mA, 100 ms pulses, close to the remote's 9 ms leader plus 60 ms of data at tens of mA.

In [ ]:

```
# --- Energizer CR2032 ---
CELL_NOMINAL_MAH = 235.0  # datasheet, to 2.0 V at 15 kOhm
# Handbook p5, 21 °C, 75 Ohm pulse 0.1 s on / 4.9 s off (read by eye)
OCV_CAP_MAH = np.array([0, 10, 25, 50, 100, 150, 175, 185, 195, 203, 208,
212])
OCV_V = np.array([2.88, 2.86, 2.88, 2.88, 2.88, 2.87, 2.85, 2.82, 2.77,
2.70, 2.62, 2.55])
IR_CAP_MAH = np.array([0, 10, 25, 50, 75, 100, 125, 150, 170, 185, 195,
202, 207, 212])
IR_OHM = np.array([9, 10, 13, 14, 16, 15, 15, 16, 18, 22, 30, 40, 55, 75])
SELF_DISCHARGE_PER_YEAR = 0.01  # handbook p7

# --- 0 °C, from handbook p4 (1 mA continuous, 0 °C vs 21 °C) ---
COLD_CAP_SCALE = 150 / 228  # 2.0 V reached at ~150 mAh instead of ~228 mAh
COLD_OCV_OFFSET = -0.20  # plateau ~2.65 V instead of ~2.85 V at 1 mA
COLD_IR_SCALE = 2.0  # ASSUMPTION: no published CR2032 pulse-IR vs
temperature found
COLD_VF_SHIFT = 0.04  # TSAL6200 -1.8 mV/K x 21 K

# --- Circuit ---
R7 = 15.0  # LED series resistor
Q1_RDS = 0.05  # AO3400A, ~48 mOhm at VGS 2.5 V
# TSAL6200 I-V, datasheet Fig. 4 (25 °C, 100 us pulses, read by eye);
# anchored to the table: 1.35 V typ at 100 mA, 2.2 V typ at 1 A
LED_IV_V = np.array([1.15, 1.20, 1.27, 1.35, 1.50, 1.70, 2.00, 2.20])
LED_IV_MA = np.array([1, 10, 30, 100, 200, 400, 700, 1000])
R8 = 47.0  # LED reservoir feed resistor
C4_C6_DC_BIAS_FACTOR = 0.60  # Samsung DC-bias curve: -40 % at 3 V
C_LED = 3 * 100e-6 * C4_C6_DC_BIAS_FACTOR
C_VDD = 10e-6 * 0.8 + 2 * 100e-9  # C1 10 uF 25 V part, light derating at 3
V; C2, C3
CARRIER_DUTY = 1 / 3

# --- Ceramic leakage: Samsung spec IR >= 50 MOhm.uF (or 10 GOhm, whichever
smaller) ---
IR_SPEC_MOHM_UF = 50.0
LEAKY_CAPS_UF = [100, 100, 100, 10]  # C4-C6, C1 (C1 assumed same spec
class)
V_CELL_TYP = 2.9

# --- MCU (CH32L103 datasheet) ---
I_RUN = 2.14e-3  # run, 8 MHz HSI, PLL off, peripherals on (Table 3-6)
I_STANDBY = 0.37e-6  # standby, no RAM retention, LSI off (Table 3-8-1)
T_BOOT_S = 6.5e-3  # reset duration after standby wake (tRSTTEMPO typ)
T_FIRMWARE_OVERHEAD_S = 10e-3 + 20e-3 + 2 * 100e-6  # debounce + release
settle + PVD settles

# --- Brown-out thresholds (PVD levels, CH32L103 datasheet Table 3-4) ---
PVD_TX_OK = 2.35  # PLS=011 rising edge; conservative for the check-before-
send
PVD_ABORT_OPTIONS = {"1.87 V (PLS=001)": 1.87, "2.08 V (PLS=010)": 2.08}
```

```
PVD_ABORT_FIRMWARE = "2.08 V (PLS=010)"  # what fw/remote.c uses
(PVD_LEVEL_ABORT)
VDD_MCU_MIN = 1.8

# --- Usage assumptions ---
PRESSES_PER_DAY = 20
REPEATS_PER_PRESS = 3  # ~0.4 s average hold
NEC_REPEAT_PERIOD_S = 0.108
```

Inputs

21 °C. OCV and internal resistance against capacity used come from the handbook's p5 chart, read by eye.

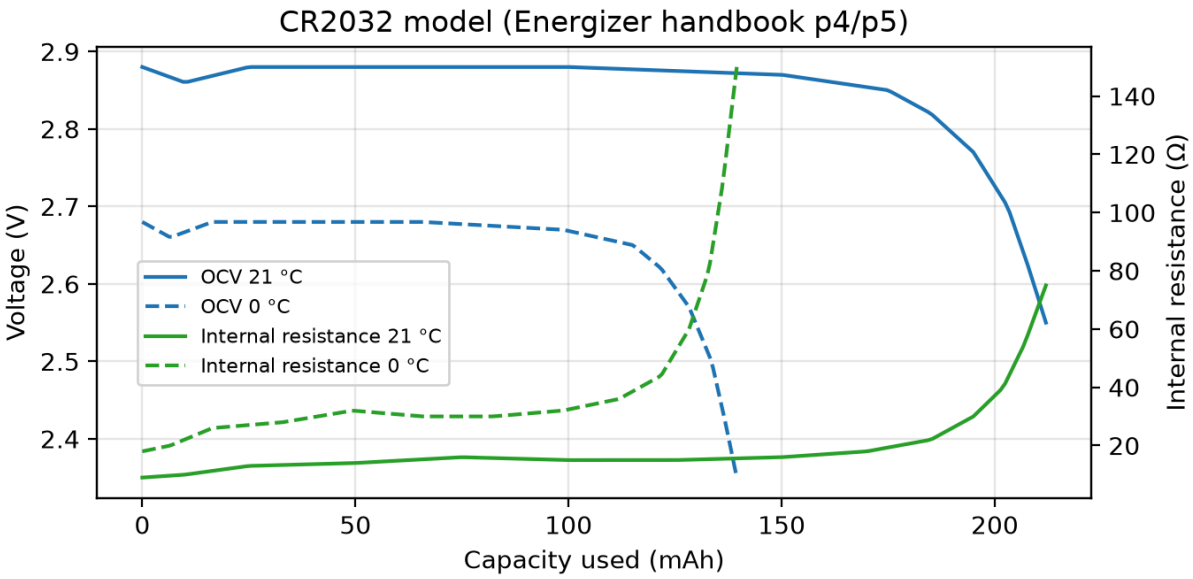
0 °C. The handbook (p4) shows a CR2032 at 1 mA running about 0.2 V lower at 0 °C (≈2.65 V instead of ≈2.85 V) and reaching 2.0 V at about 150 mAh instead of about 228 mAh. The cold case therefore:

- compresses the capacity axis by 0.66
- shifts the voltage down by 0.20 V
- multiplies the internal resistance by 2 (**assumption**: I found no published CR2032 pulse-resistance-vs-temperature data)
- shifts the LED I–V curve up by 0.04 V (TSAL6200 tempco)

Circuit (schematic as of this notebook): the MCU runs straight off the cell, and the LED runs from its own reservoir C4–C6 refilled through R8. C4–C6 keep 60% of their 100 µF at 3 V (Samsung DC-bias curve). The LED current is averaged over the 38 kHz carrier (1/3 duty). That's fine for the supply nodes because the reservoir and R8 smooth the carrier ripple.

In [ ]:
```
CASES = {
    "21 °C": dict(cap_scale=1.0, ocv_offset=0.0, ir_scale=1.0,
vf_shift=0.0),
    "0 °C": dict(
        cap_scale=COLD_CAP_SCALE,
        ocv_offset=COLD_OCV_OFFSET,
        ir_scale=COLD_IR_SCALE,
        vf_shift=COLD_VF_SHIFT,
    ),
}

def cell_model(life_mah, cap_scale, ocv_offset, ir_scale):
    """OCV and internal resistance at each capacity-used point."""
    voc = np.interp(life_mah, OCV_CAP_MAH * cap_scale, OCV_V) + ocv_offset
    rint = np.interp(life_mah, IR_CAP_MAH * cap_scale, IR_OHM) * ir_scale
    return voc, rint
```



LED drive

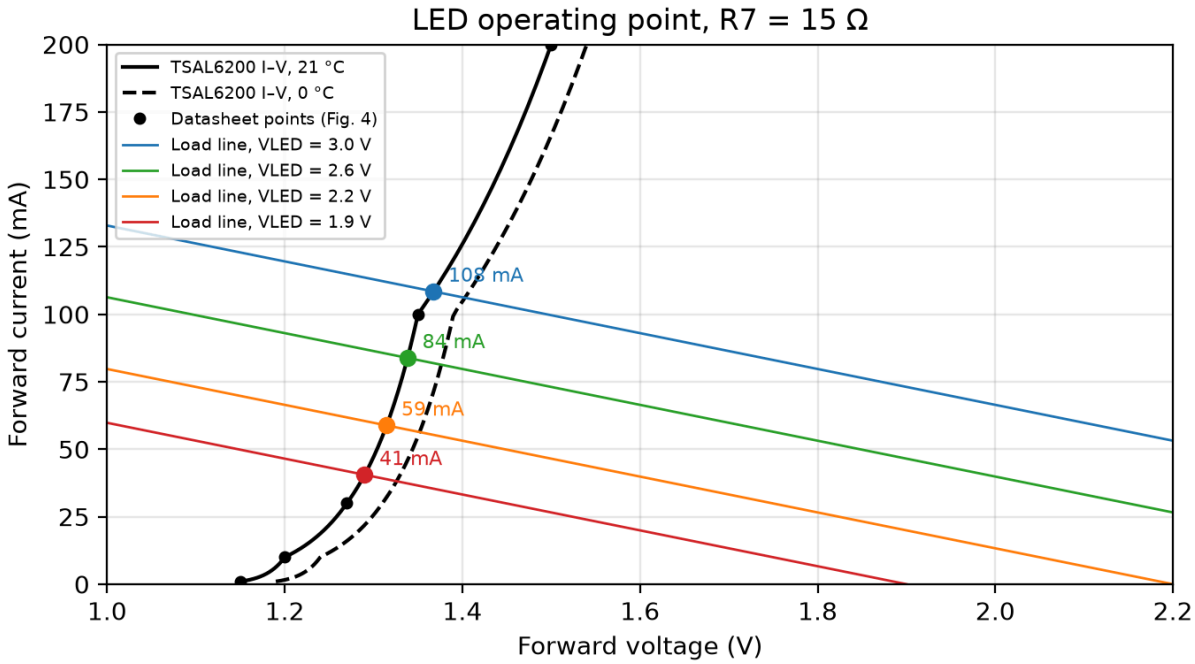
During a carrier pulse the LED, R7 and Q1 sit across the LED reservoir VLED, so the pulse current is where the LED's I–V curve meets the load line VLED − I·(R7 + $R_{DS(on)}$). Using the datasheet curve rather than a fixed V_F matters at low VLED, where the knee (≈1.2 V) makes the current fall off faster than a straight line. Radiant intensity is roughly proportional to current (TSAL6200 Fig. 5) and range goes as its square root, so relative range ≈ √(I / 100 mA).

No saturation in this design. In the TSAL6200 datasheet, Fig. 5 (radiant intensity vs current) and Fig. 6 (radiant power vs current) are close to linear from 1 mA to 1 A. The table gives 72 mW/sr typ at 100 mA and 600 mW/sr at 1 A, so 10× the current gives ~8.3× the light: ~15–20 % droop, and only near 1 A. The LED here runs at 40–170 mA, well inside the linear part. Self-heating is negligible at this duty cycle; Fig. 7 gives −0.6 %/K, so it's ~15 % *brighter* at 0 °C than at 25 °C, which isn't modelled here. Because light tracks LED current, and the battery supplies that same current, light per unit of battery charge is roughly independent of R7.

In [ ]:
```
_I_GRID = np.logspace(-4, 0, 600)  # 0.1 mA .. 1 A

def led_vf(i_amps, vf_shift=0.0):
    """TSAL6200 forward voltage, interpolated in log(I)."""
    return np.interp(np.log10(i_amps * 1e3), np.log10(LED_IV_MA), LED_IV_V)
+ vf_shift

def led_current(vled, vf_shift=0.0, r7=R7):
    """LED pulse current (A) for a given VLED: I–V curve meets the R7 + Q1
load line."""
    v_needed = led_vf(_I_GRID, vf_shift) + _I_GRID * (r7 + Q1_RDS)
    return np.interp(vled, v_needed, _I_GRID, left=0.0)
```



Supply simulation

Two nodes: **VDD** (MCU, C1–C3) is fed from the cell's open-circuit voltage through its internal resistance; **VLED** (C4–C6) is fed from VDD through R8 and drains into the LED during marks. The sequence is what one held press sends: an NEC frame, then two repeat codes on the 108 ms grid. It's simulated at every point of the cell's life at once, starting from a rested cell.

In [ ]:

```
def nec_frame(command, address=0x00):
    """(duration_s, is_mark) segments for one NEC frame."""
    data = address | ((~address & 0xFF) << 8) | (command << 16) |
((~command & 0xFF) << 24)
    segs = [(9e-3, True), (4.5e-3, False)]
    for i in range(32):
        segs += [(563e-6, True), (1688e-6 if (data >> i) & 1 else 563e-6,
False)]
    segs.append((563e-6, True))
    return segs

def nec_repeat():
    return [(9e-3, True), (2.25e-3, False), (563e-6, True)]

def pad_to_period(segs, period=NEC_REPEAT_PERIOD_S):
    """Append the idle gap up to the next 108 ms slot."""
    used = sum(d for d, _ in segs)
    return segs + [(period - used, False)]

# One held press: frame + 2 repeats, each window padded to 108 ms
PRESS_WINDOWS = [
    pad_to_period(nec_frame(0x02)),
    pad_to_period(nec_repeat()),
```

```
    pad_to_period(nec_repeat()),
]
FRAME_S = sum(d for d, _ in nec_frame(0x02))
```

In [ ]:
```
def simulate(voc, rint, vf_shift, windows, dt=10e-6, r8=R8, c_led=C_LED,
r7=None):
    """Simulate supply nodes for every (voc, rint) pair at once.

    r8 and c_led may be scalars or arrays matching voc (for sweeps); r7
    overrides the schematic's LED resistor (scalar).

    Returns per-life-point minimum VDD and minimum VLED, plus the LED
    charge drawn in each window (coulombs, per life point).
    """
    vdd = voc.astype(float).copy()
    vled = vdd.copy()
    vdd_min = vdd.copy()
    vled_min = vled.copy()
    q_windows = []
    for window in windows:
        q = np.zeros_like(vdd)
        for duration, mark in window:
            for _ in range(int(round(duration / dt))):
                i_led = (
                    CARRIER_DUTY * led_current(vled, vf_shift, *(() if r7
is None else (r7,)))
                    if mark
                    else 0.0
                )
                i_cell = (voc - vdd) / rint
                i_r8 = (vdd - vled) / r8
                vdd = vdd + (i_cell - i_r8 - I_RUN) * dt / C_VDD
                vled = vled + (i_r8 - i_led) * dt / c_led
                np.minimum(vdd_min, vdd, out=vdd_min)
                np.minimum(vled_min, vled, out=vled_min)
                q = q + i_led * dt
        q_windows.append(q)
    return vdd_min, vled_min, q_windows
```

In [ ]:
```
sims = {}
for _name, _p in CASES.items():
    _life = np.arange(0.0, OCV_CAP_MAH[-1] * _p["cap_scale"], 0.5)
    _voc, _rint = cell_model(_life, _p["cap_scale"], _p["ocv_offset"],
_p["ir_scale"])
    _vdd_min, _vled_min, _q = simulate(_voc, _rint, _p["vf_shift"],
PRESS_WINDOWS)
    sims[_name] = {
        "life_mah": _life,
        "vdd_rest": _voc - I_RUN * _rint,  # awake, LED off: what the TX-OK
check sees
        "vdd_min": _vdd_min,
        "vled_min": _vled_min,
        "q_frame": _q[0],
        "q_repeat": np.mean(_q[1:], axis=0),
    }
```

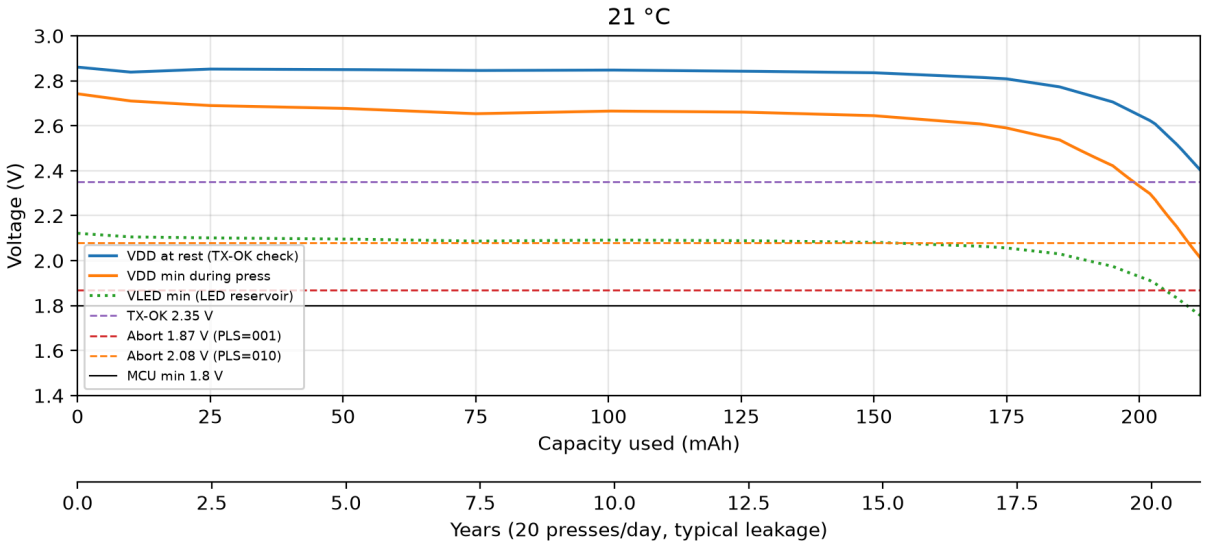
In [ ]:

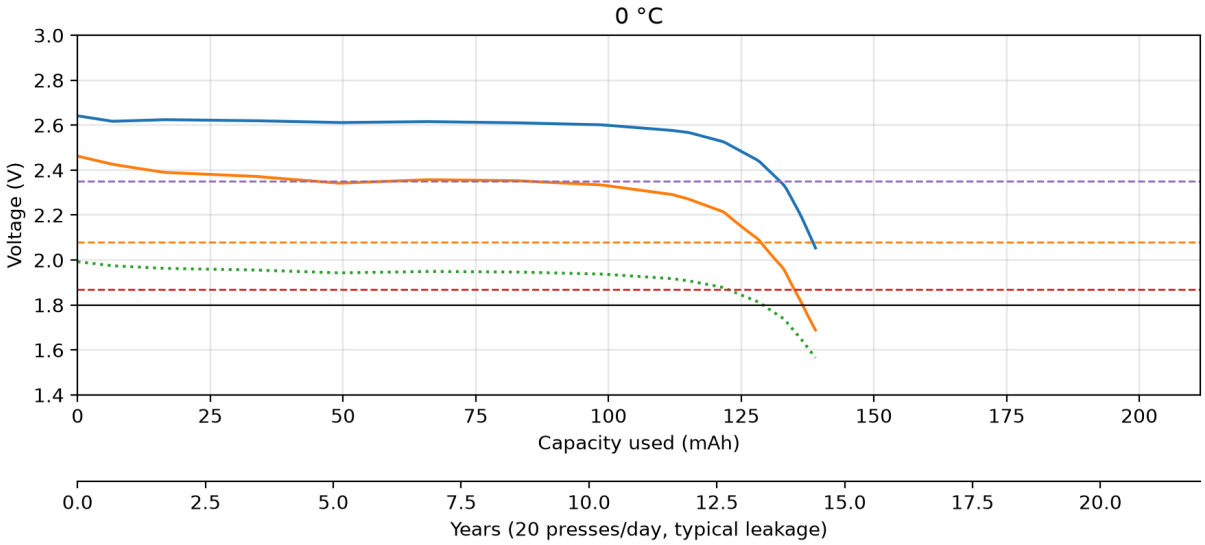
```
MAH = 3.6  # coulombs per mAh

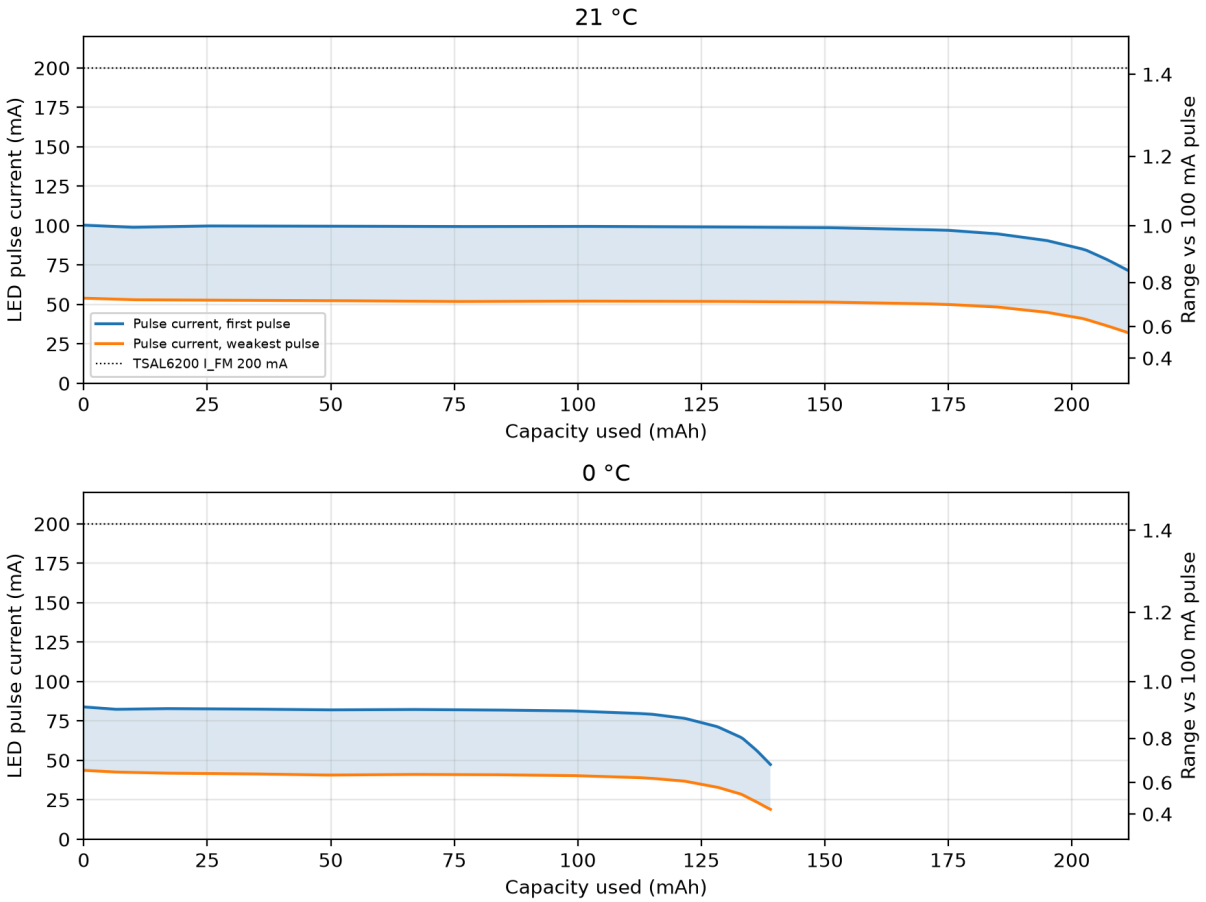
# Ceramic leakage at the spec's guaranteed-minimum insulation resistance
i_leak_spec = sum(V_CELL_TYP / (min(IR_SPEC_MOHM_UF / c, 10_000) * 1e6) for
c in LEAKY_CAPS_UF)
LEAKAGE = {"typical": 0.0, "spec max": i_leak_spec}

standby_mah_per_year = I_STANDBY * 8760 * 1000
self_discharge_mah_per_year = SELF_DISCHARGE_PER_YEAR * CELL_NOMINAL_MAH
t_awake = T_BOOT_S + T_FIRMWARE_OVERHEAD_S + FRAME_S + REPEATS_PER_PRESS *
NEC_REPEAT_PERIOD_S

consumption = {}
for _name, _s in sims.items():
    _q_led = _s["q_frame"][0] + REPEATS_PER_PRESS * _s["q_repeat"][0]
    _q_press = _q_led + I_RUN * t_awake
    _press_mah_per_year = PRESSES_PER_DAY * 365 * _q_press / MAH
    for _leak_name, _i_leak in LEAKAGE.items():
        _per_year = (
            _press_mah_per_year
            + standby_mah_per_year
            + self_discharge_mah_per_year
            + _i_leak * 8760 * 1000
        )
        consumption[(_name, _leak_name)] = {
            "q_led": _q_led,
            "q_press": _q_press,
            "mah_per_year": _per_year,
            "mah_per_day": _per_year / 365,
        }
```


21 °C
Voltage (V)
3.0
2.8
2.6
2.4
2.2
2.0
1.8
1.6
1.4
VDD at rest (TX-OK check)
VDD min during press
VLED min (LED reservoir)
TX-OK 2.35 V
Abort 1.87 V (PLS=001)
Abort 2.08 V (PLS=010)
MCU min 1.8 V
0
25
50
75
100
125
150
175
200
Capacity used (mAh)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Years (20 presses/day, typical leakage)


0 °C
Voltage (V)
3.0
2.8
2.6
2.4
2.2
2.0
1.8
1.6
1.4
0
25
50
75
100
125
150
175
200
Capacity used (mAh)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Years (20 presses/day, typical leakage)



End of life and battery life

End of life is the first point where a press would be refused (VDD at rest below the TX-OK level) or cut short (VDD during the press below the abort level). The capacity used up to there is what's usable.

Charge per press = LED charge (frame + repeats, from the simulation of a fresh cell) + MCU run current × time awake (boot, debounce, the frames themselves, release settle). Each year also loses standby current, the cell's self-discharge and capacitor leakage.

Capacitor leakage is shown two ways. *Typical*: modern X5R parts usually leak far less than their guarantee, so it's taken as negligible. *Spec max*: Samsung only guarantees insulation resistance ≥ 50 MΩ·µF, i.e. 0.5 MΩ for each 100 µF part, so leakage could legally be several µA. That's 10× the MCU's standby current, so measure a real board.

In [ ]:
```
def usable_mah(s, abort_v):
    """Capacity used at end of life, and which check ends it."""
    refused = s["vdd_rest"] < PVD_TX_OK
    aborted = s["vdd_min"] < abort_v
    dead = refused | aborted
    if not dead.any():
        return s["life_mah"][-1], "cell exhausted (end of handbook data)"
    i = np.argmax(dead)
```

```
    return s["life_mah"][i], "abort in frame" if aborted[i] else "TX-OK at
rest"

results = []
for _name, _s in sims.items():
    for _label, _v in PVD_ABORT_OPTIONS.items():
        _usable, _limit = usable_mah(_s, _v)
        for _leak in LEAKAGE:
            _c = consumption[(_name, _leak)]
            results.append(
                {
                    "case": _name,
                    "abort": _label,
                    "leakage": _leak,
                    "usable_mah": _usable,
                    "usable_pct": 100 * _usable / CELL_NOMINAL_MAH,
                    "limited_by": _limit,
                    "q_press_mc": 1e3 * _c["q_press"],
                    "q_led_mc": 1e3 * _c["q_led"],
                    "presses": _usable * MAH / _c["q_press"],
                    "mah_per_year": _c["mah_per_year"],
                    "years": _usable / _c["mah_per_year"],
                }
            )
```

Temp	Abort level	Cap leakage	Usable (of 235 mAh)	Limited by	Charge per press	Presses	mAh/year	Years
21 °C	1.87 V (PLS=001)	typical	212 mAh (90 %)	cell exhausted (end of handbook data)	2.23 mC (1.31 LED)	341,457	10.1	20.9
21 °C	1.87 V (PLS=001)	spec max	212 mAh (90 %)	cell exhausted (end of handbook data)	2.23 mC (1.31 LED)	341,457	167.6	1.3
21 °C	**2.08 V (PLS=010) ◀ firmware**	**typical**	**210 mAh (89 %)**	**abort in frame**	**2.23 mC (1.31 LED)**	**338,228**	**10.1**	**20.7**
21 °C	2.08 V (PLS=010) ◀ firmware	spec max	210 mAh (89 %)	abort in frame	2.23 mC (1.31 LED)	338,228	167.6	1.2
0 °C	1.87 V (PLS=001)	typical	132 mAh (56 %)	TX-OK at rest	1.99 mC (1.08 LED)	239,279	9.6	13.8
0 °C	1.87 V (PLS=001)	spec max	132 mAh (56 %)	TX-OK at rest	1.99 mC (1.08 LED)	239,279	167.1	0.8
0 °C	**2.08 V (PLS=010) ◀ firmware**	**typical**	**129 mAh (55 %)**	**abort in frame**	**1.99 mC (1.08 LED)**	**232,959**	**9.6**	**13.4**
0 °C	2.08 V (PLS=010) ◀ firmware	spec max	129 mAh (55 %)	abort in frame	1.99 mC (1.08 LED)	232,959	167.1	0.8

Highlighted: the expected battery life, with the firmware's abort level (2.08 V (PLS=010)) and typical capacitor leakage. Assumes 20 presses/day, each a frame plus 3 repeats, all at the stated temperature. Fixed drains: standby 3.2 mAh/year, self-discharge 2.4 mAh/year, capacitor leakage at spec max 18.0 µA (158 mAh/year).

Sweep: R8 and LED reservoir size

The LED reservoir sags through each frame, so late pulses are weaker than the first. More capacitance (more 100 µF parts in parallel) or a smaller R8 flattens it. But a smaller R8 lets the LED pull harder on the cell, which drags VDD down and ends the cell's useful life sooner, especially when cold.

For each combination, the sweep simulates one NEC frame across the cell's life and reports:

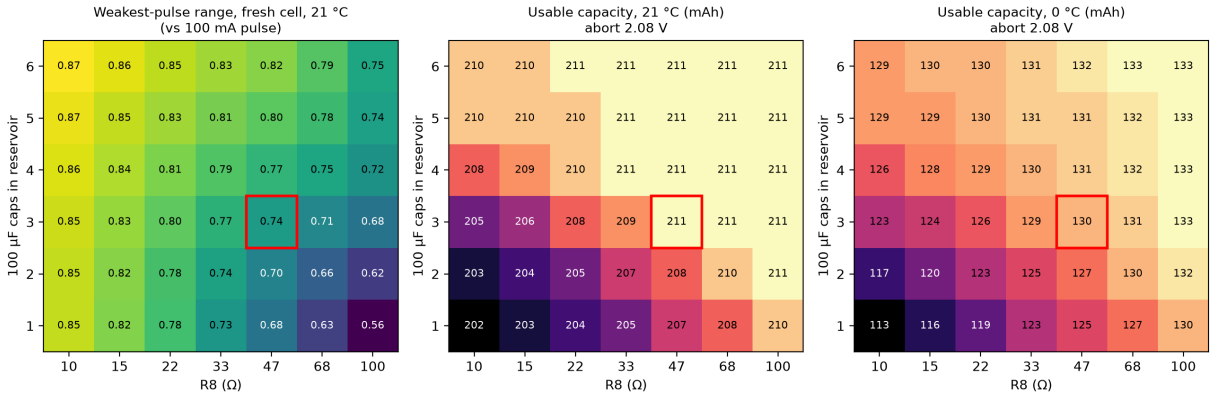
- **weakest-pulse range**: range of the weakest pulse in a frame from a fresh cell at 21 °C, relative to a 100 mA pulse (√I scaling)
- **usable capacity** at 21 °C and 0 °C, with the 2.08 V abort level

The current design (R8 = 47 Ω, 3 caps) is outlined.

In [ ]:
```
R8_SWEEP = np.array([10, 15, 22, 33, 47, 68, 100])
N_CAPS_SWEEP = np.arange(1, 7)  # 100 uF 1206 parts in parallel
SWEEP_ABORT_V = 2.08

sweep = {}
for _name, _p in CASES.items():
    _life = np.arange(0.0, OCV_CAP_MAH[-1] * _p["cap_scale"], 1.0)
    _voc, _rint = cell_model(_life, _p["cap_scale"], _p["ocv_offset"],
_p["ir_scale"])
    _shape = (len(R8_SWEEP), len(N_CAPS_SWEEP), len(_life))
    _r8 = np.broadcast_to(R8_SWEEP[:, None, None], _shape).ravel()
    _c = np.broadcast_to(N_CAPS_SWEEP[None, :, None] * 100e-6 *
C4_C6_DC_BIAS_FACTOR, _shape).ravel()
    _v = np.broadcast_to(_voc, _shape).ravel()
    _r = np.broadcast_to(_rint, _shape).ravel()
    # One frame is the worst case; repeats are shorter
    _vdd_min, _vled_min, _ = simulate(_v, _r, _p["vf_shift"],
PRESS_WINDOWS[:1], r8=_r8, c_led=_c)
    _vdd_min = _vdd_min.reshape(_shape)
    _vled_min = _vled_min.reshape(_shape)
    _vdd_rest = np.broadcast_to(_voc - I_RUN * _rint, _shape)

    _dead = (_vdd_rest < PVD_TX_OK) | (_vdd_min < SWEEP_ABORT_V)
    _usable = np.where(_dead.any(axis=2), _life[np.argmax(_dead, axis=2)],
_life[-1])
    _i_weak = led_current(_vled_min[:, :, 0], _p["vf_shift"])
    sweep[_name] = {
        "usable_mah": _usable,
        "weak_range": np.sqrt(_i_weak / 0.1),
        "weak_ma": _i_weak * 1e3,
    }
```



R8	Caps		Weakest pulse, fresh 21 °C (range)	Usable 21 °C (mAh)	Usable 0 °C (mAh)
47 Ω	3	current design	54 mA (0.74)	211	130
22 Ω	3	smaller R8, no extra parts	64 mA (0.80)	208	126
22 Ω	4	one more cap, smaller R8	66 mA (0.81)	210	129
47 Ω	5	two more caps, same R8	63 mA (0.80)	211	131
10 Ω	1	almost straight off the cell	72 mA (0.85)	202	113

The first pulse of every frame is the same (~100 mA fresh) in all cases. Only the tail of the frame changes.

Caveats

- **Chart data is read by eye.** The end-of-life point sits on the steep part of the internal-resistance curve, so ±5 mAh of reading error moves it by about the same.
- **The cold model is part measured, part assumed.** The capacity and voltage shifts come from Energizer's 1 mA curves. The ×2 on internal resistance is an assumption; below 0 °C it will be worse.
- **No margin on the PVD levels.** The datasheet gives typical thresholds only.
- **Capacitor leakage is the big unknown.** At the guaranteed minimum it would dominate everything else and cut life to a year or two. Real parts are usually far better, but the spec doesn't promise it. If a prototype shows µA-level leakage, a P-FET to disconnect C4–C6 in standby fixes it.
- **Shelf life caps it anyway.** Energizer quotes a 10-year shelf life, so treat anything past ~10 years as "the battery lasts as long as a CR2032 can".

Appendix: one button press in time

NEC timing. A full frame is a 9 ms leader burst, a 4.5 ms space, 32 bits and a 562.5 µs stop burst: 68.1 ms, the same for every code because each byte is sent with its inverse. Each bit is a 562.5 µs burst followed by a 562.5 µs space (0) or a 1687.5 µs space (1). While the button stays held, repeat codes (9 ms burst, 2.25 ms space, 562.5 µs burst = 11.8 ms) follow. Every transmission starts on a **108 ms** grid, start to start. That leaves ~39.9 ms of silence after the full frame and ~96.2 ms between repeats.

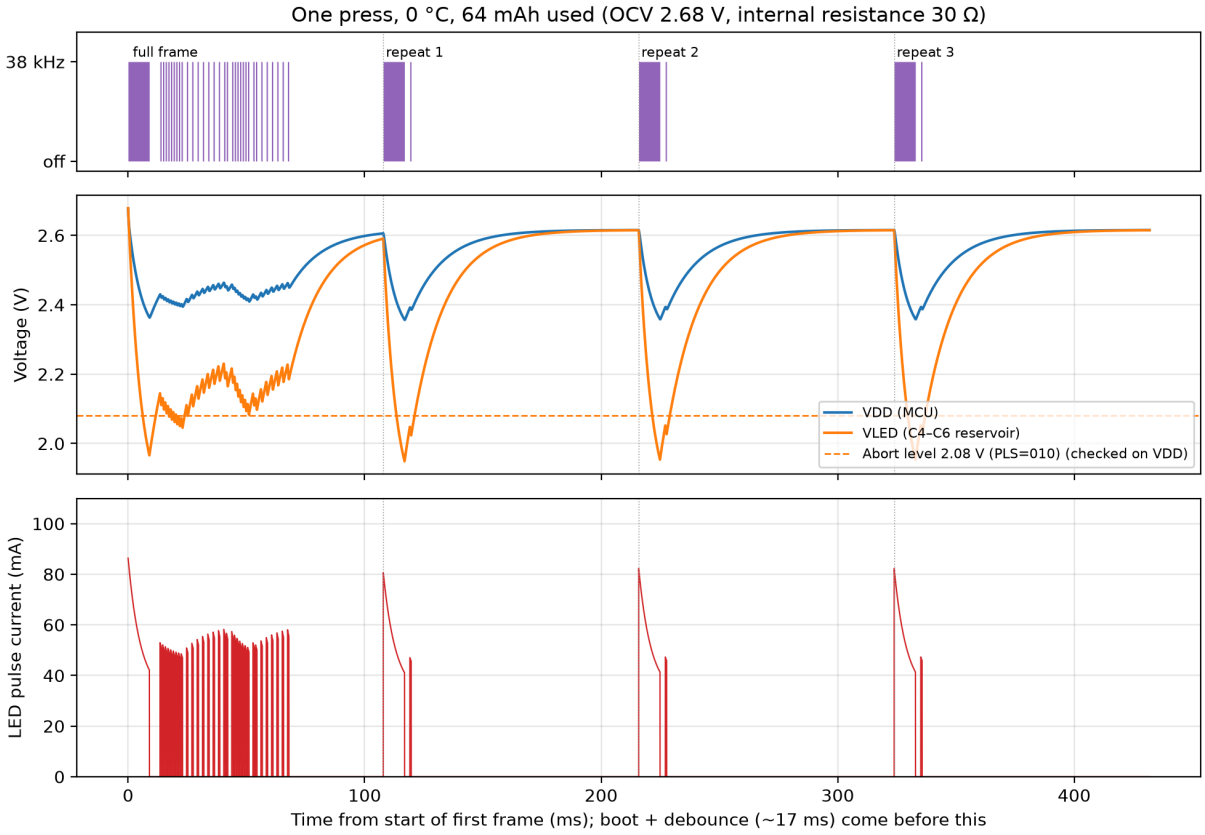
Element	Burst	Space	Total
Leader	9 ms	4.5 ms	13.5 ms
Bit 0 / bit 1	562.5 µs	562.5 µs / 1687.5 µs	1.125 / 2.25 ms
Full frame (incl. stop burst)			68.1 ms
Repeat code	9 ms + 562.5 µs	2.25 ms	11.8 ms
Frame / repeat period			108 ms

Below is a typical press (the full frame plus the repeats assumed in the usage model) from a cell at **0 °C, halfway through its usable life**. The 38 kHz carrier isn't drawn. LED current is the height of each carrier pulse; the average during a burst is a third of that.

In [ ]:
```
TRACE_CASE = "0 °C"
_p = CASES[TRACE_CASE]
_usable = next(
    r["usable_mah"]
    for r in results
    if r["case"] == TRACE_CASE and r["abort"] == PVD_ABORT_FIRMWARE and
r["leakage"] == "typical"
)
trace_life_mah = 0.5 * _usable
_voc, _rint = cell_model(np.array([trace_life_mah]), _p["cap_scale"],
_p["ocv_offset"], _p["ir_scale"])
_voc, _rint = float(_voc[0]), float(_rint[0])

# Same supply model as simulate(), but for one life point, recording every
step
_segs = pad_to_period(nec_frame(0x02))
for _ in range(REPEATS_PER_PRESS):
    _segs += pad_to_period(nec_repeat(), NEC_REPEAT_PERIOD_S)
_dt = 10e-6
_vdd = _vled = _voc
_t, _mark, _vdd_t, _vled_t, _ipk_t = [], [], [], [], []
_now = 0.0
for _duration, _is_mark in _segs:
    for _ in range(int(round(_duration / _dt))):
        _ipk = float(led_current(np.array([_vled]), _p["vf_shift"])[0]) if
_is_mark else 0.0
        _i_cell = (_voc - _vdd) / _rint
        _i_r8 = (_vdd - _vled) / R8
        _vdd += (_i_cell - _i_r8 - I_RUN) * _dt / C_VDD
        _vled += (_i_r8 - CARRIER_DUTY * _ipk) * _dt / C_LED
```

```
        _t.append(_now)
        _mark.append(_is_mark)
        _vdd_t.append(_vdd)
        _vled_t.append(_vled)
        _ipk_t.append(_ipk)
        _now += _dt
trace = {
    "t_ms": np.array(_t) * 1e3,
    "mark": np.array(_mark, dtype=float),
    "vdd": np.array(_vdd_t),
    "vled": np.array(_vled_t),
    "i_pk_ma": np.array(_ipk_t) * 1e3,
    "voc": _voc,
    "rint": _rint,
}
```



Appendix B: choosing R7

The single-press trace shows the LED current falling from ~86 mA to ~42 mA during the 9 ms leader, as the reservoir drains. Would a bigger R7 help? It lowers the first pulse, but it also drains the reservoir more slowly. The question is which R7 gives the strongest *weakest* pulse, since that pulse limits range. In a press it's the end of the leader.

Model

During one burst of length T, take:

- **Source.** The cell and R8 form a Thévenin source: $U_0 = V_{oc} - R_{int} I_{run}$ behind $R_s = R_{int} + R_8$. VDD's own 10 µF is ignored (its time constant is ~0.2 ms, much less than T).
- **LED.** Linearised as $V_F + r_d I$ around the operating point. The pulse path is $R = R_7 + R_{DS(on)} + r_d$.
- **Carrier.** With duty d, the reservoir C supplies an average of $d\,i$, where i is the pulse current.

With $x = v_{LED} - V_F$ and $U = U_0 - V_F$:

$$C\,\dot{x} = \frac{U - x}{R_s} - \frac{d\,x}{R}, \qquad x(0) = U \text{ (rested)}$$

which is first order, so

$$x(t) = x_\infty + (U - x_\infty)\,e^{-t/\tau}, \qquad x_\infty = U\frac{R}{R+a}, \qquad \tau = C\,\frac{R_s R}{R+a}, \qquad a = d\,R_s$$

and the pulse current $i = x/R$ is

$$\boxed{i(t) = \frac{U}{R+a}\left[1 + \frac{a}{R}\,e^{-t/\tau}\right]}$$

Limits: a smaller R7 wins

- First pulse: $i(0) = U/R$.
- A very long burst: $i(\infty) = U/(R+a)$.

Both strictly decrease with R. So for very short or very long bursts, a smaller R7 is always better, and making R7 *bigger* can't help.

Finite burst: an optimum exists, but has no closed form

Substitute $s = 1/R$. Then $T/\tau = k + \beta s$ with $k = T/(CR_s)$ and $\beta = d\,T/C$, giving

$$\frac{i(T)}{U} = \frac{s}{1+as}\left[1 + a\,e^{-k}\,s\,e^{-\beta s}\right]$$

$$\frac{\partial}{\partial s}\frac{i(T)}{U} = \frac{1 + a\,e^{-k}\,s\,e^{-\beta s}\left[(2-\beta s)(1+as) - as\right]}{(1+as)^2}$$

The optimum is where the numerator is zero. That equation mixes a polynomial with $e^{-\beta s}$, and the quadratic factor stops it reducing to the Lambert W function. **There's no elementary closed form for the optimal R7**, but it's a trivial one-dimensional numerical solve.

Closed-form bounds

1. **Optimum is below R_{crit}.** A root needs the bracket to be negative, i.e. $2 + (a - \beta)s - a\beta s^2 < 0$. In terms of R:

$$R_{opt} < R_{crit} = \frac{\beta - a + \sqrt{(a-\beta)^2 + 8a\beta}}{4}$$

   Above R_{crit}, a smaller R7 always gives a stronger weakest pulse. $\beta = dT/C$ is the resistance at which the reservoir would drain in about one burst.

2. **Pulse-current limit.** A fresh cell (up to ~3.2 V open-circuit, handbook p3/p5) must not push the first pulse past the TSAL6200's 200 mA peak rating:

$$R_7 \geq R_{FM} = \frac{V_{max} - V_F(I_{FM})}{I_{FM}} - R_{DS(on)}$$

So the best R7 is $\max\left(R_{FM},\ R_{opt}\right)$ with $R_{opt} < R_{crit}$. It also costs battery: a smaller R7 draws more charge per press and sags VDD further. That's checked against the full nonlinear simulation below.

Decision: R7 stays at 15 Ω for now. 10 Ω would give ~12 % stronger weakest pulses (~6 % more range) for ~1–1.5 years less calculated life, still above the cell's 10-year shelf life.

In [ ]:
```
T_LEADER_S = 9e-3
LED_I_FM = 0.200  # TSAL6200 peak forward current (tp/T = 0.5, tp = 100 us)
V_FRESH_MAX = 3.2  # fresh CR2032 OCV before first load (handbook p3/p5)

# LED linearised through the 30 mA and 100 mA datasheet points
_i1, _i2 = 0.030, 0.100
R_D = (led_vf(_i2) - led_vf(_i1)) / (_i2 - _i1)
V_F0 = led_vf(_i1) - R_D * _i1

def i_burst(t, r7, voc, rint, vf_shift=0.0):
    """Closed-form pulse current after t seconds of a burst (A)."""
    u = voc - rint * I_RUN - (V_F0 + vf_shift)
    rs = rint + R8
    r = r7 + Q1_RDS + R_D
    a = CARRIER_DUTY * rs
    tau = C_LED * rs * r / (r + a)
    return u / (r + a) * (1 + a / r * np.exp(-t / tau))

def r_crit(rint):
    """Upper bound on the optimal total pulse-path resistance R (Ohm)."""
    a = CARRIER_DUTY * (rint + R8)
    beta = CARRIER_DUTY * T_LEADER_S / C_LED
    return (beta - a + np.sqrt((a - beta) ** 2 + 8 * a * beta)) / 4

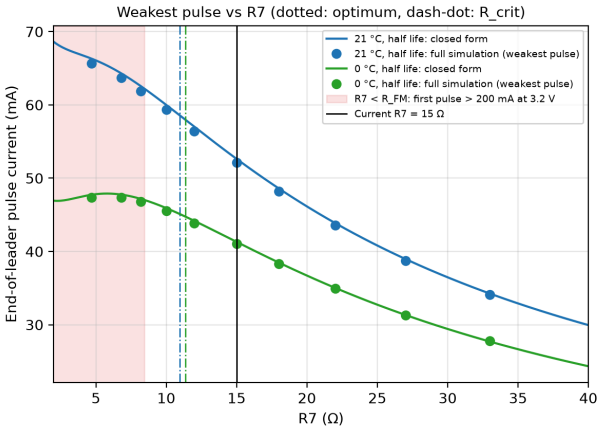
def r7_opt_numeric(voc, rint, vf_shift=0.0):
    """Numerical maximiser of the end-of-leader current over R7."""
    r7 = np.linspace(0.5, 60, 4000)
    return r7[np.argmax(i_burst(T_LEADER_S, r7, voc, rint, vf_shift))]

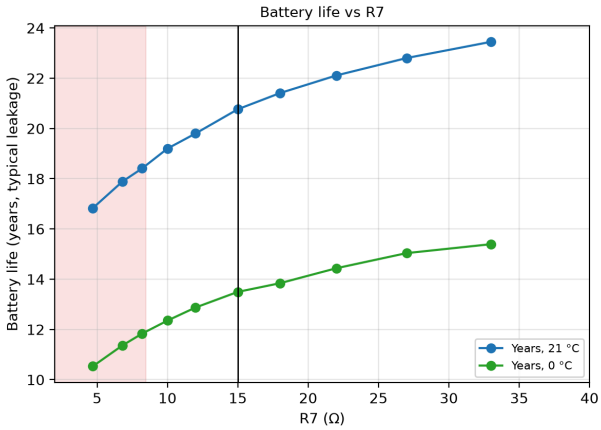
R7_FM = (V_FRESH_MAX - led_vf(LED_I_FM)) / LED_I_FM - Q1_RDS
```

In [ ]:
```
R7_SWEEP = np.array([4.7, 6.8, 8.2, 10, 12, 15, 18, 22, 27, 33])
_abort_v = PVD_ABORT_OPTIONS[PVD_ABORT_FIRMWARE]

r7_sweep = {}
for _name, _p in CASES.items():
    _life = np.arange(0.0, OCV_CAP_MAH[-1] * _p["cap_scale"], 2.0)
    _voc, _rint = cell_model(_life, _p["cap_scale"], _p["ocv_offset"],
_p["ir_scale"])
    _vdd_rest = _voc - I_RUN * _rint
    _rows = []
    for _r7 in R7_SWEEP:
        _vdd_min, _vled_min, _q = simulate(_voc, _rint, _p["vf_shift"],
PRESS_WINDOWS, r7=_r7)
        _dead = (_vdd_rest < PVD_TX_OK) | (_vdd_min < _abort_v)
        _usable = _life[np.argmax(_dead)] if _dead.any() else _life[-1]
        _half = np.argmin(np.abs(_life - _usable / 2))
        _q_press = _q[0][0] + REPEATS_PER_PRESS * np.mean([_w[0] for _w in
_q[1:]]) + I_RUN * t_awake
        _per_year = PRESSES_PER_DAY * 365 * _q_press / MAH +
standby_mah_per_year + self_discharge_mah_per_year
        _rows.append(
            {
                "r7": _r7,
                "usable_mah": _usable,
```

```
                "years": _usable / _per_year,
                "first_ma_fresh": 1e3 * led_current(_vdd_rest[:1],
_p["vf_shift"], _r7)[0],
                "weak_ma_half": 1e3 * led_current(_vled_min[_half : _half +
1], _p["vf_shift"], _r7)[0],
                "voc_half": _voc[_half],
                "rint_half": _rint[_half],
            }
        )
    r7_sweep[_name] = _rows
```





R7	First pulse, fresh 21 °C	Weakest pulse, half life 21 °C	Weakest pulse, half life 0 °C	Years 21 °C	Years 0 °C
4.7 Ω	269 mA ⚠	66 mA	47 mA	16.8	10.5
6.8 Ω	199 mA ⚠	64 mA	47 mA	17.9	11.4
8.2 Ω	169 mA	62 mA	47 mA	18.4	11.8
10 Ω	143 mA	59 mA	46 mA	19.2	12.4
12 Ω	122 mA	56 mA	44 mA	19.8	12.9
15 Ω ◀ current	100 mA	52 mA	41 mA	20.8	13.5
18 Ω	84 mA	48 mA	38 mA	21.4	13.8
22 Ω	70 mA	44 mA	35 mA	22.1	14.4
27 Ω	57 mA	39 mA	31 mA	22.8	15.0
33 Ω	47 mA	34 mA	28 mA	23.5	15.4

The first pulse here is at the fresh cell's loaded OCV. A brand-new cell can briefly sit at ~3.2 V, which is what R_{FM} = 8.4 Ω guards against. ⚠ = within 10 % of the 200 mA rating. LED linearisation: V_F = 1.24 V + 1.14 Ω × I.